MO809: Final Presentation

Cooperative Perception with DiscoNet Under Perfect Communication

Learning Distilled Collaboration Graph for Multi-Agent Perception

Li et al., NeurIPS 2021

LUIS ALBERTO VÁSQUEZ VARGAS,
Wellington Lobato, Guilherme Maia,
Allan M. de Souza, Edmundo Madeira

November 5, 2026

Agenda

01 **Motivation**

02 **Problem**

03 **Project Objective**

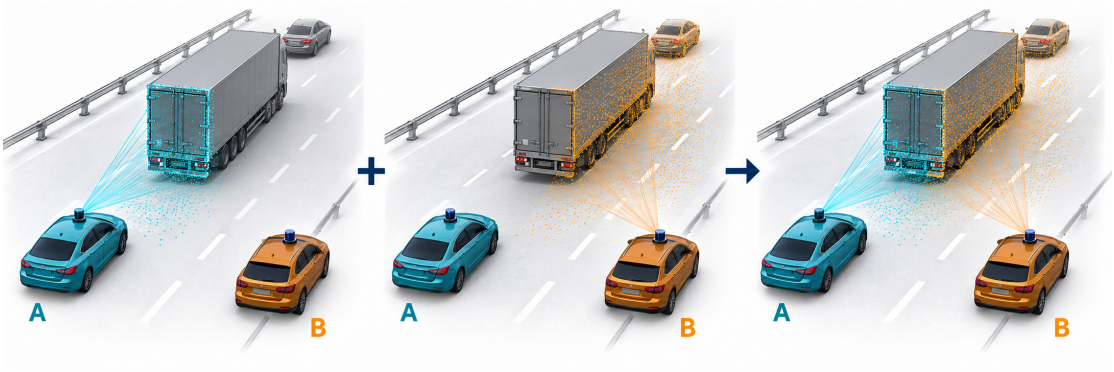
04 **Project Development**

05 **Evaluation**

06 **Results**

Motivation

Cooperative Perception

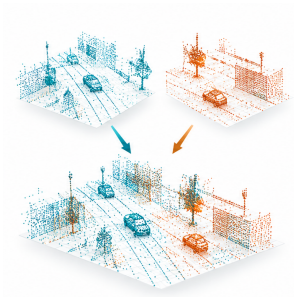


Seeing Beyond One View

Source: AI generated illustration.

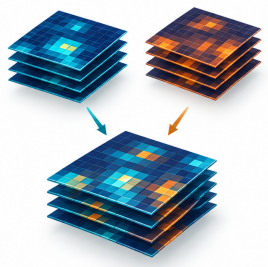
Collaboration Strategies

Early Fusion



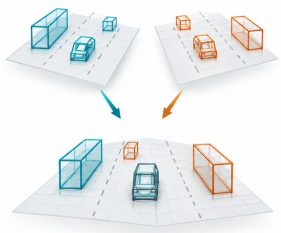
Raw Points
Before Encoding

Intermediate Fusion



Learned Features
Before Detection

Late Fusion



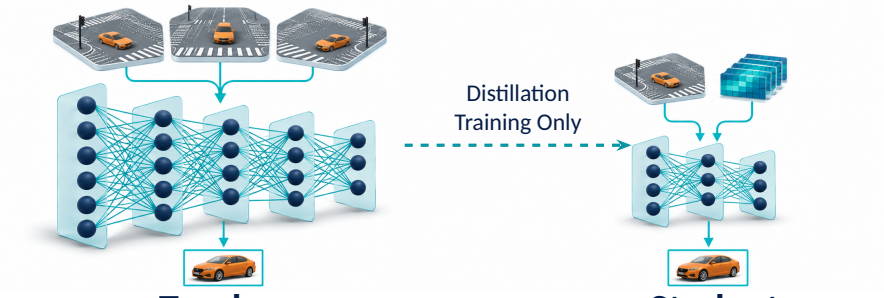
Boxes + Scores
After Detection

Source: AI generated illustrations.

Problem

Problem

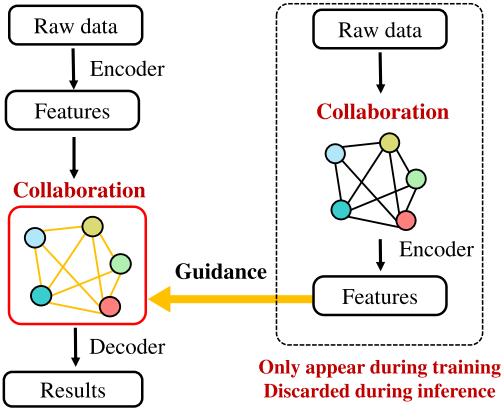
Can a Holistic Early-Fusion Model Be Compressed While Preserving Detection Accuracy?



Source: AI generated illustrations.

DiscoNet Collaboration Strategy

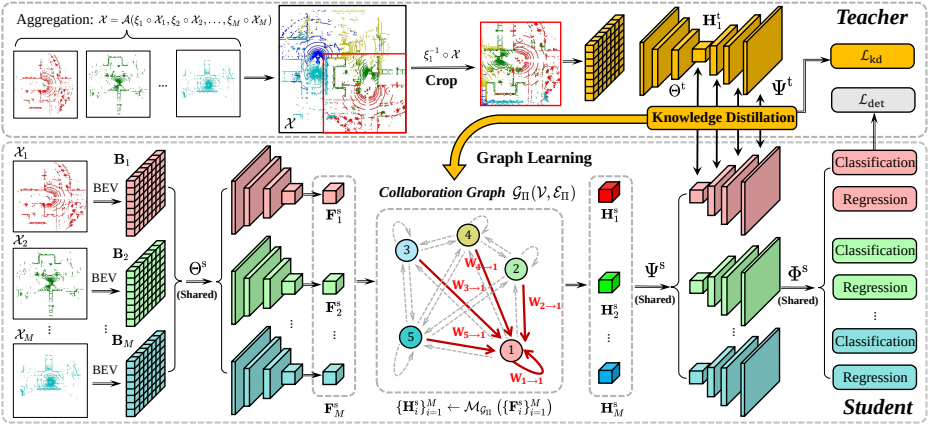
Student
Intermediate
Collaboration



Teacher
Early
Collaboration

Source: Li et al., DiscoNet, Figure 1(c).

DiscoNet Architecture

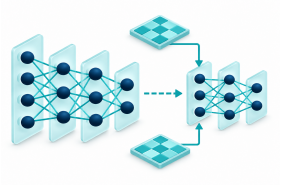


Source: Li et al., DiscoNet, Figure 2.

Project Objective

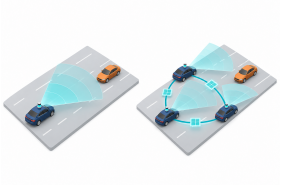
Project Objective

Evaluate DiscoNet's Detection Quality and Scalability under Perfect Communication



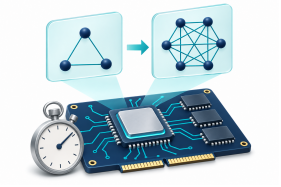
Understand

Teacher–Student Pipeline and Feature Fusion



Compare

Individual vs. Cooperative Detection



Benchmark

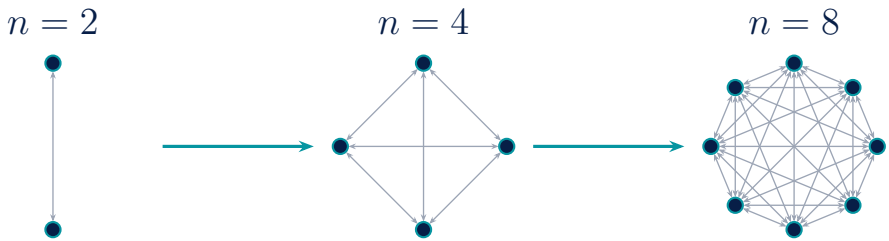
Accuracy and Cost as n Increases

n = Number of Cooperating Agents

Source: AI generated illustrations.

Our Proposal: Accuracy and Scale

Benchmark the Limits of Full Collaboration

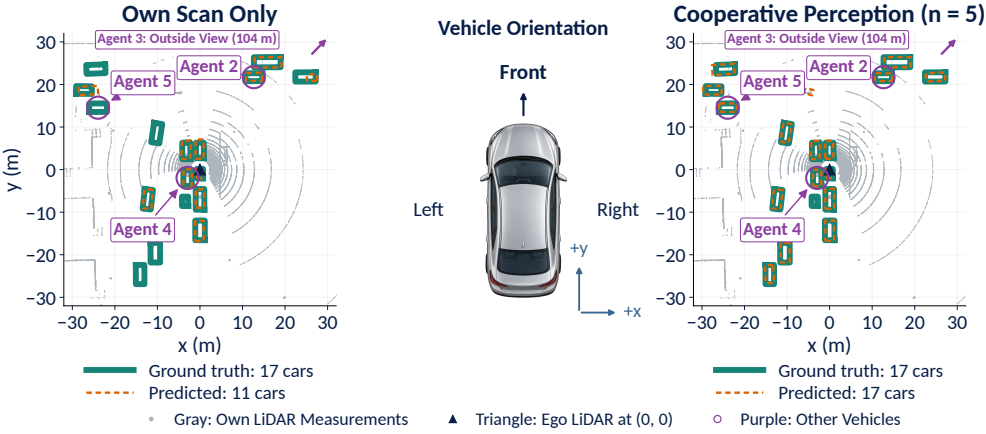


2 Directed Peer Pairs 12 Directed Peer Pairs 56 Directed Peer Pairs

Quadratic Pairwise Work: $n(n-1) = \mathcal{O}(n^2)$

Measure AP and Time Inference

Local vs. Cooperative Detection



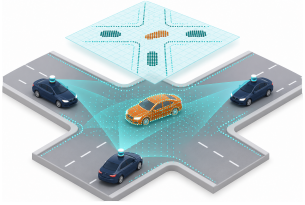
One Example: Scene 5, Raw Frame 000066; Ego Vehicle 1

Source: AI generated illustrations.

Project Development

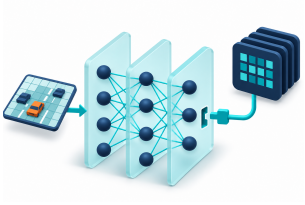
Project Development

Data



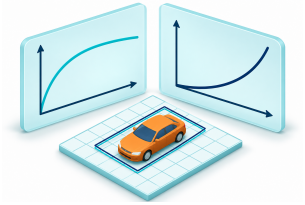
V2X-Sim 2.0

Models



Compatible Checkpoints

Evidence

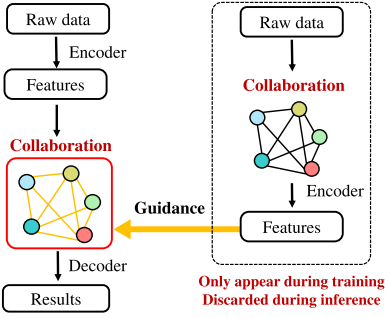


AP + Scaling Curves

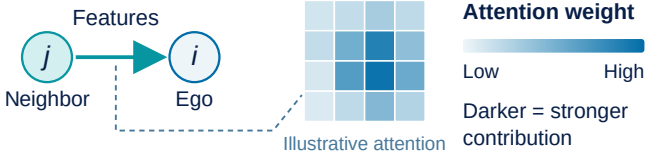
Source: AI generated illustrations.

How DiscoNet Works

Teacher + Student



Spatial Attention

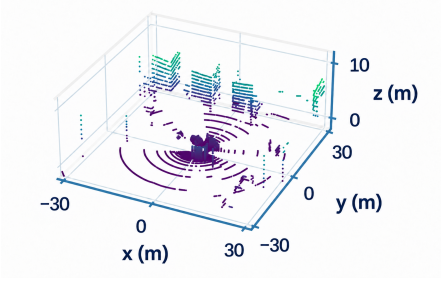


Share Features Weight by Location

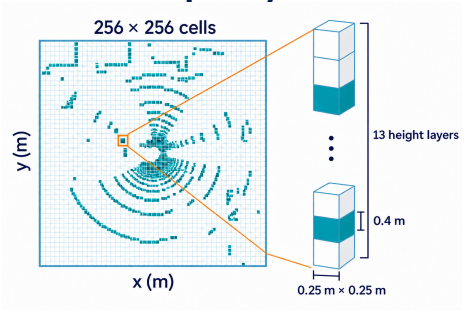
Source: Li et al., Fig. 1(c). AI generated attention illustration.

LiDAR Preprocessing

Point Cloud



Occupancy Grid

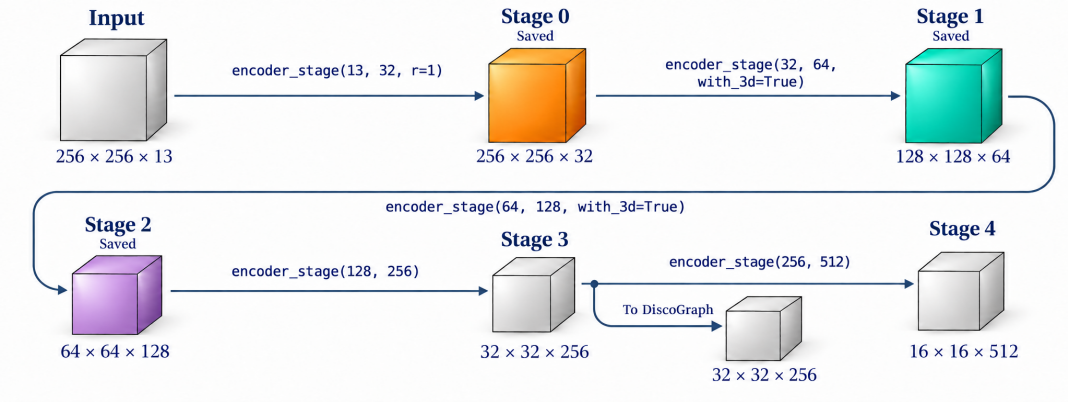


$$256 \times 256 \times 13$$

1 = Occupied 0 = No Retained Return

Source: CoPerception voxelization. AI generated illustrations.

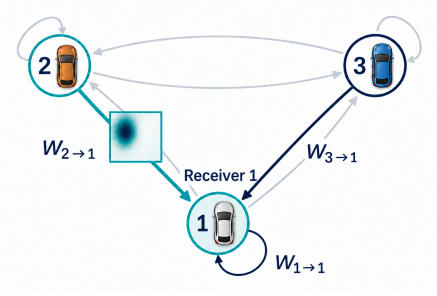
Encoder Pipeline

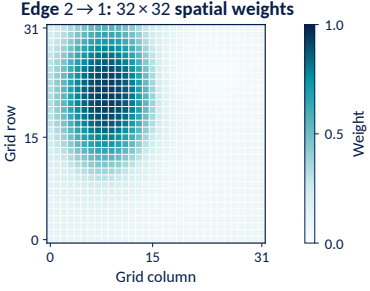


H = Height W = Width C = Channels

Source: CoPerception, `LidarEncoder` (adapted). AI generated illustration.

DiscoGraph: Spatial Collaboration

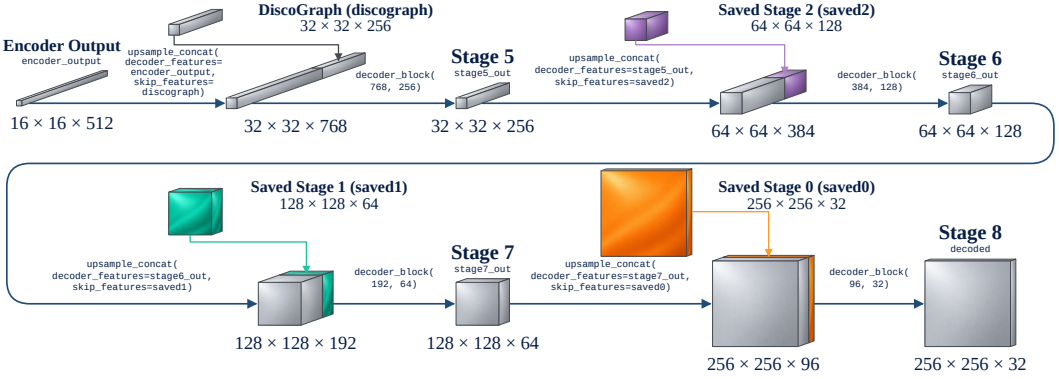




$$\sum_u w_{u\to v}(r,c) \quad = \quad 1$$

Source: Li et al., Sec. 2.1. AI generated illustrations.

Decoder Pipeline

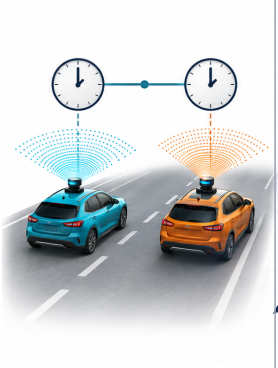
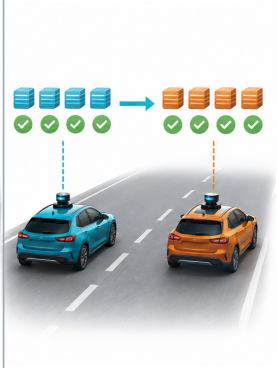


H = Height W = Width C = Channels

Source: CoPerception, `LidarDecoder` (adapted). AI generated illustration.

Evaluation

Perfect Communication

Synchronized	Accurate Poses	Zero Delay	No Packet Loss
			
Same observation time	Position and heading	0 ms latency	0% packet loss

AI-generated illustration.

Experiment Setup

Configuration of the completed 1,000-frame benchmark

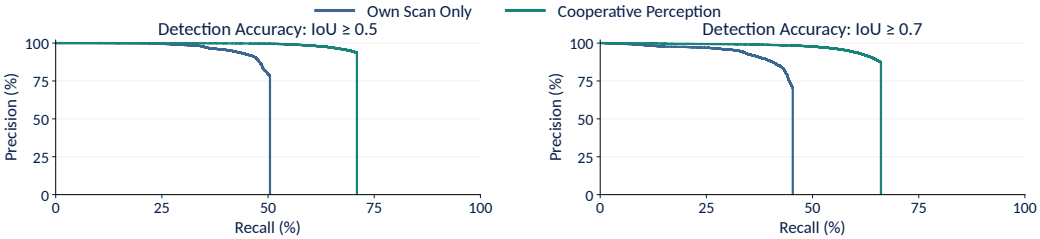
Hardware	
GPU	NVIDIA GeForce RTX 5060 Ti
GPU Memory	16 GB
CPU	AMD Ryzen 7 5700X
CPU Cores	8 cores / 16 threads
System RAM	16 GB

Software	
OS	Omarchy (Linux)
Python	3.14.8
PyTorch	2.9.1+cu128
CUDA Runtime	12.8

Source: Recorded benchmark configuration, November 3.

Cooperation Improves Detection

Condition	AP@0.5 (%)	AP@0.7 (%)
Own Scan Only	49.10	43.05
Cooperative Perception	**70.41**	**64.82**



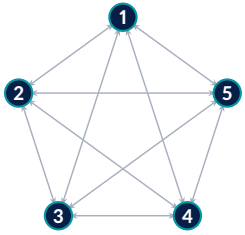
Sharing Adds 21.31 / 21.77 AP Percentage Points

Source: Li et al., DiscoNet (method).

Results

Original vs. Tree Collaboration

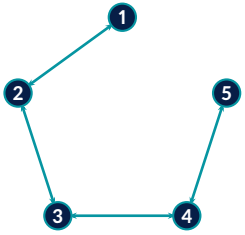
Original: Complete Graph



n^2 Directed Edges

Kruskal Selection

Tree: Maximum Spanning Tree



$3n - 2$ Directed Edges

Source: Own elaboration.

Runtime and Detection AP

n	Original Median (ms)	Tree Median (ms)	Original AP@0.5 (%)	Tree AP@0.5 (%)	Original AP@0.7 (%)	Tree AP@0.7 (%)
1	7.82	7.86	48.91	48.91	45.54	45.54
2	13.34	13.25	60.94	60.94	53.67	53.67
3	21.99	21.41	66.52	60.17	58.71	52.34
4	31.39	29.38	70.80	58.06	63.39	50.37
5	41.57	36.54	74.46	62.65	67.53	53.66
6	52.43	43.63	73.54	57.71	66.61	50.69
7	66.67	52.89	73.93	57.87	66.70	50.57
8	79.83	60.47	73.80	59.16	66.90	52.01
9	92.07	67.48	74.50	61.56	67.06	53.33
10	102.80	70.12	74.46	61.72	67.53	53.66
11	118.76	76.93	74.00	58.57	67.05	50.44
12	133.49	82.84	74.17	58.77	67.20	50.72
13	147.97	86.52	74.36	59.50	67.18	51.61
14	165.79	92.73	74.90	60.93	67.50	52.75
15	**185.05**	**99.25**	**74.46**	**60.48**	**67.53**	**52.45**

Source: Own elaboration.

Runtime and Accuracy Trade-Off

At $n = 15$ Original $\rightarrow$ Tree

Inference Time	AP@0.5	AP@0.7
46.4%	**-13.98**	**-15.08**
Less Inference Time	Percentage Points	Percentage Points
185.05 $\rightarrow$ 99.25 ms	74.46% $\rightarrow$ 60.48%	67.53% $\rightarrow$ 52.45%

Lower Runtime Comes with Lower Detection AP

Source: Own elaboration.

Thank You

Questions?

Appendix

Appendix Contents

A	Architecture and Implementation	A3–A26
B	Evaluation and Reproduction	A27–A37
C	Conv2D and Conv3D	A38–A58
D	Complete Experiment Record	A59–A79
E	Image Generation Example	A80–A83

Appendix A

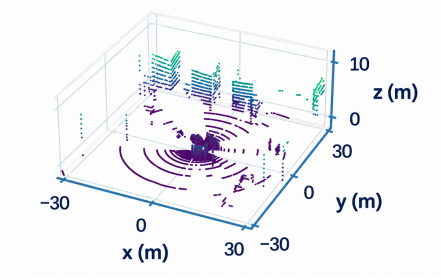
Architecture and Implementation

Voxelization, reusable blocks, encoder code, spatial alignment, attention, fusion, and decoding.

LiDAR Preprocessing

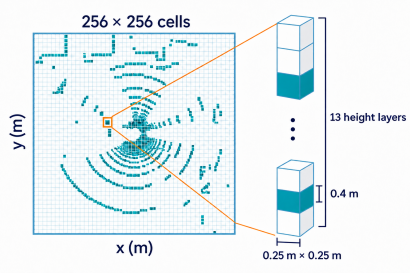
Preprocessing converts point clouds stored in PCD format into fixed-size occupancy grids for DiscoNet.

LiDAR point cloud



One LiDAR scan per vehicle, preprocessed independently.

Fixed-size occupancy grid



Each cell stores occupancy in 13 height layers.
Teal: occupied (1).
White: not occupied (0).

Source: AI generated illustrations

Voxelization

`voxelize_occupy`: `PointCloud` $\longrightarrow$ `OccupancyGrid3D`

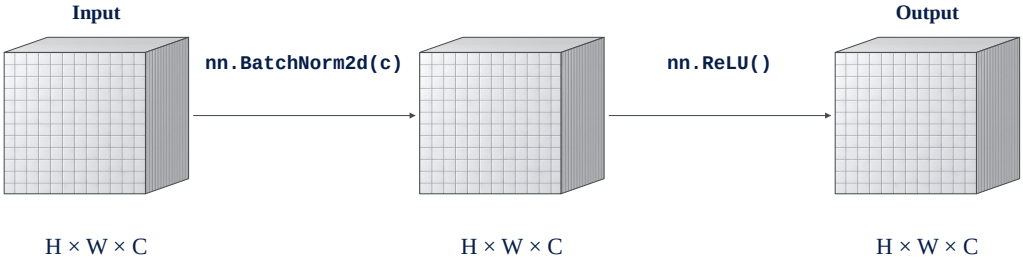
Units: meters

1. **Validate Coordinates**
   Validate 3D coordinates; reject missing or infinite values.
2. **Remove points near the origin**
   In the sensor frame, remove points where $-1 < x < 1$ **AND** $-1 < y < 1$.
3. **Crop the scan**
   Keep points satisfying $-32 < x < 32$ **AND** $-32 < y < 32$ **AND** $-3 < z < 2$.
4. **Start with zeros**
   Create a grid of 256×256 cells with 13 height layers, all starting at 0.
5. **Fill the grid**
   For each retained point, find its $0.25 \times 0.25 \times 0.4$ voxel and set `grid[i,j,k] = 1`.
   Cells without points stay at 0.

Source: CoPerception, `voxelize_occupy` — github.com/coperception/coperception

Normalization and Activation

`norm_relu(c)` c = number of channels




```
def norm_relu(c):
    return nn.Sequential(nn.BatchNorm2d(c), nn.ReLU())
```

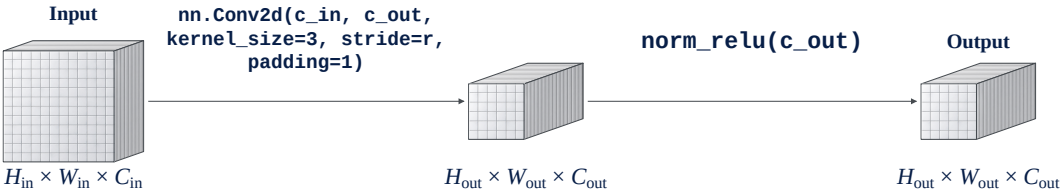
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

2D Unit

`unit_2d(c_in, r, c_out)`




```
def unit_2d(c_in, r, c_out):
    conv = nn.Conv2d(c_in, c_out, kernel_size=3, stride=r, padding=1)
    return nn.Sequential(conv, *norm_relu(c_out))
```

Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

2D Unit: Variants

Each variant reuses `unit_2d`.

Initial: Set Channels

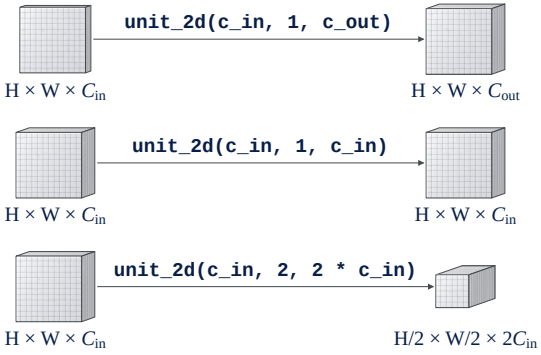
```
def unit_2d_initial(c_in, c_out):
    return unit_2d(c_in, 1, c_out)
```

Keep: Preserve Shape

```
def unit_2d_keep(c_in):
    return unit_2d(c_in, 1, c_in)
```

Down: Halve H and W, Double C

```
def unit_2d_down(c_in):
    return unit_2d(c_in, 2, 2 * c_in)
```



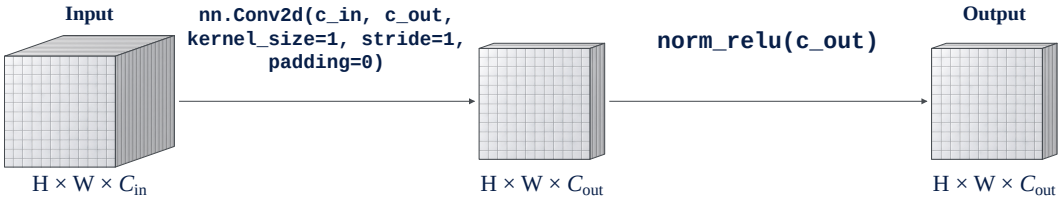
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustrations

2D Pointwise Unit

`unit_2d_pointwise(c_in, c_out)`




```python
def unit_2d_pointwise(c_in, c_out):
    conv = nn.Conv2d(c_in, c_out, kernel_size=1, stride=1, padding=0)
    return nn.Sequential(conv, *norm_relu(c_out))
```

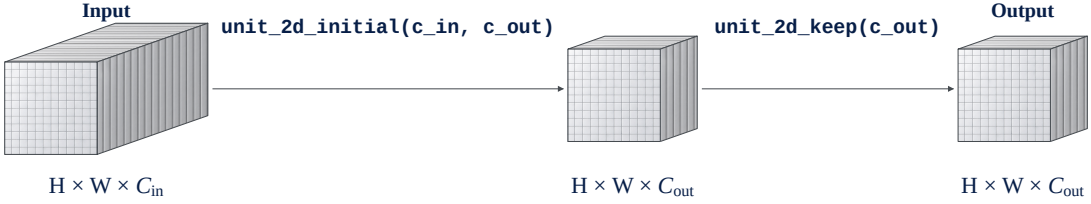
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

Decoder Block

`decoder_block(c_in, c_out)`




```
def decoder_block(c_in, c_out):
    return nn.Sequential(unit_2d_initial(c_in, c_out),
                         unit_2d_keep(c_out))
```

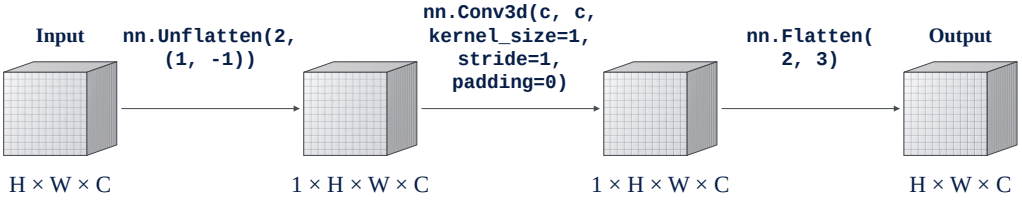
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

3D Convolution

unit_3d(c)




```python
def unit_3d(c):
    unflatten = nn.Unflatten(2, (1, -1))
    conv = nn.Conv3d(c, c, kernel_size=1, stride=1, padding=0)
    flatten = nn.Flatten(2, 3)
    return nn.Sequential(unflatten, conv, flatten)
```

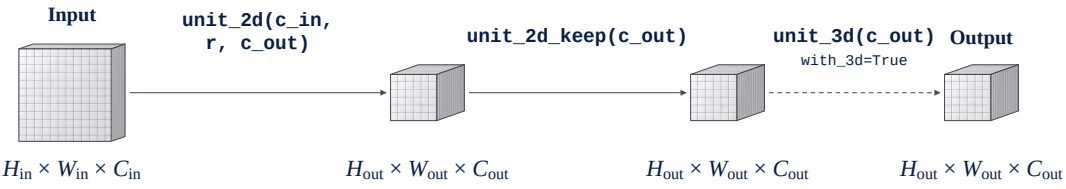
Tensor shape: $D \times H \times W \times C$

D = depth (1), H = height, W = width, C = channels

Source: AI generated illustration

Encoder Stage

`encoder_stage(c_in, c_out, r=2, with_3d=False)`




```python
def encoder_stage(c_in, c_out, r=2, with_3d=False):
    layers = [unit_2d(c_in, r, c_out), unit_2d_keep(c_out)]
    if with_3d:
        layers.append(unit_3d(c_out))
    return nn.Sequential(*layers)
```

Tensor shape: $H \times W \times C$

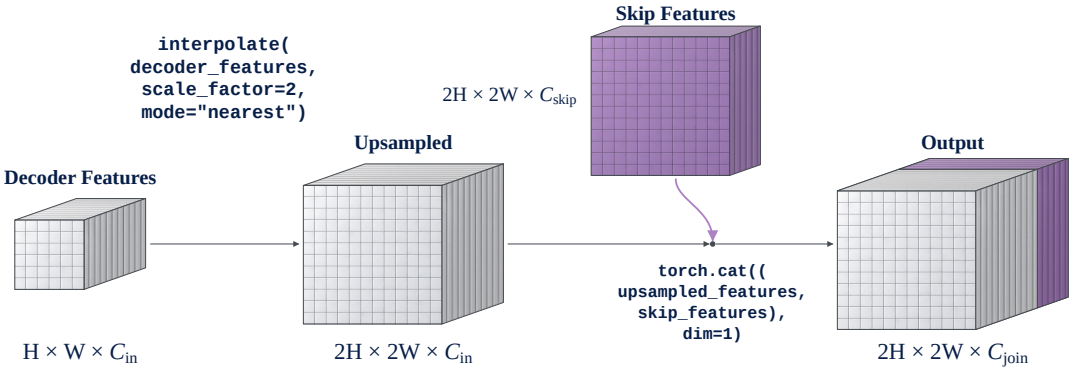
H = height, W = width, C = channels

Source: AI generated illustration

Upsample and Concat

`upsample_concat(decoder_features, skip_features)`

$$C_{\text{join}} = C_{\text{in}} + C_{\text{skip}}.$$



Tensor shape: $H \times W \times C$

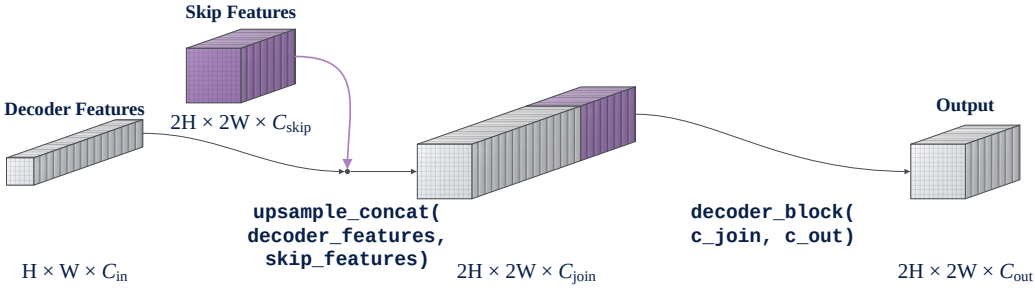
H = height, W = width, C = channels

Source: AI generated illustration

Decoder Stage

`decoder_stage(c_in, c_skip, c_out)`

$$C_{\text{join}} = C_{\text{in}} + C_{\text{skip}}.$$

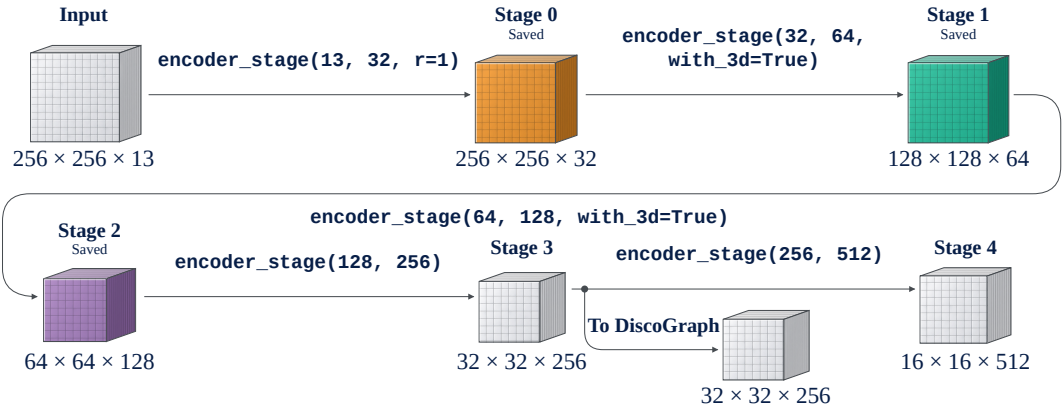


Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

Encoder Pipeline



Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

Encoder Code

Simplified PyTorch-style architecture.

Encoder

```
encoder = Sequential(
    Conv2d(13, 32, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(32),
    ReLU(),
    Conv2d(32, 32, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(32),
    ReLU(),
    Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
    BatchNorm2d(64),
    ReLU(),
    Conv2d(64, 64, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(64),
    ReLU(),
    Unflatten(2, (1, -1)),
    Conv3d(64, 64, kernel_size=1, stride=1, padding=0),
    Flatten(2, 3),
    Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
    BatchNorm2d(128),
    ReLU(),
```

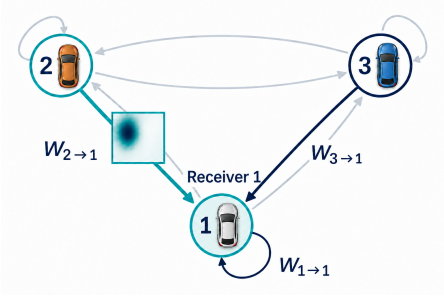
Encoder (Continued)

```
    Conv2d(128, 128, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(128),
    ReLU(),
    Unflatten(2, (1, -1)),
    Conv3d(128, 128, kernel_size=1, stride=1, padding=0),
    Flatten(2, 3),
    Conv2d(128, 256, kernel_size=3, stride=2, padding=1),
    BatchNorm2d(256),
    ReLU(),
    Conv2d(256, 256, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(256),
    ReLU(),
    Conv2d(256, 512, kernel_size=3, stride=2, padding=1),
    BatchNorm2d(512),
    ReLU(),
    Conv2d(512, 512, kernel_size=3, stride=1, padding=1),
    BatchNorm2d(512),
    ReLU(),
)
```

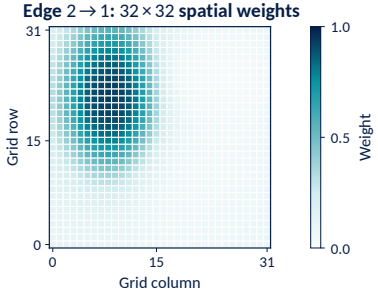
Source: Adapted from the appendix of Li et al.

DiscoGraph

For $u \to v$, cell (r, c) is a small region in receiver v's top-down feature map, in v's coordinates. Sender u's feature map is aligned to v's feature map so both describe the same physical locations.



At each cell, v combines all agents' information with normalized weights: $\sum_{u=1}^{n} w_{u\to v}(r, c) = 1$.

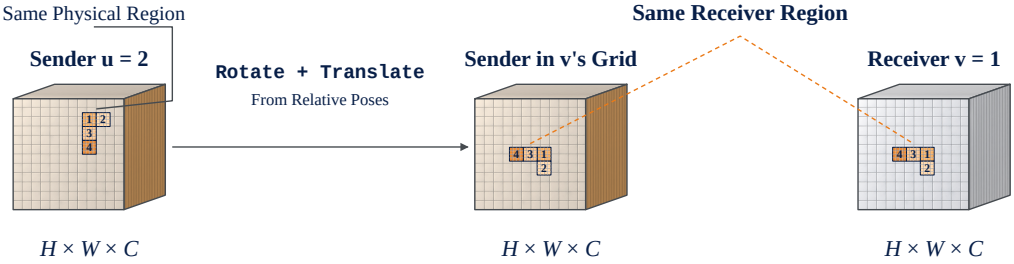


Higher weights give an agent more influence.

Source: AI generated illustration

DiscoGraph: Feature Alignment

Rotate and translate sender u's feature map into receiver v's coordinates.



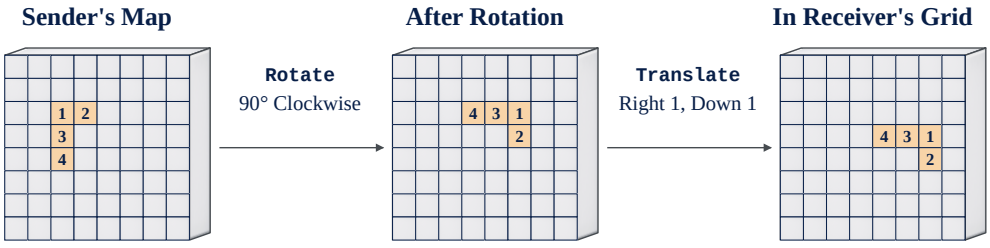
All channels use the same transform.

Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

DiscoGraph: Rotate and Translate



Example: 8×8 map, one feature channel.

Source: AI generated illustration

DiscoGraph: Alignment in PyTorch

1. `align_features` ⟶

```
import math
from torch.nn import functional as F

def align_features(
    sender_features, angle_deg, dx, dy
):
    _, C, H, W = sender_features.shape
    a = math.radians(angle_deg)
    cos, sin = math.cos(a), math.sin(a)
    transform = sender_features.new_tensor([
        [cos, -sin, 2 * dx / W],
        [sin,  cos, 2 * dy / H],
        [0,    0,    1],
    ])
```

2. `align_features` **(Continued)**

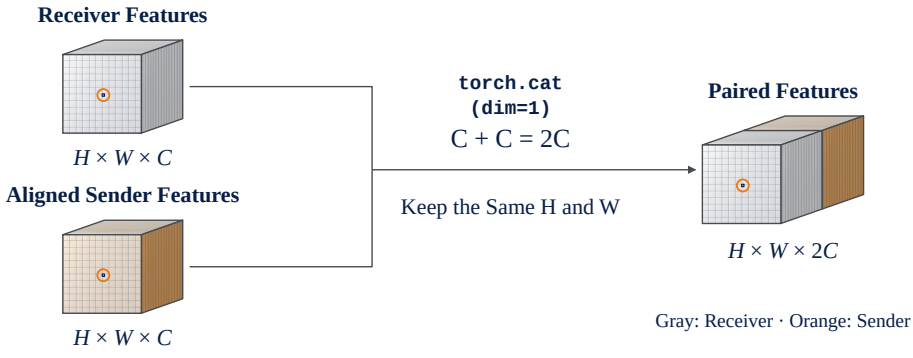
```
    inverse = transform.inverse()
    theta = inverse[:2].unsqueeze(0)
    grid = F.affine_grid(
        theta, (1, C, H, W),
        align_corners=False)
    return F.grid_sample(
        sender_features, grid,
        align_corners=False)
```

Same Example: 90°**, Right 1, Down 1**

```
aligned_features = align_features(
    sender_features, angle_deg=90, dx=1, dy=1
)
```

Source: PyTorch affine_grid and grid_sample documentation

DiscoGraph: Pair the Features

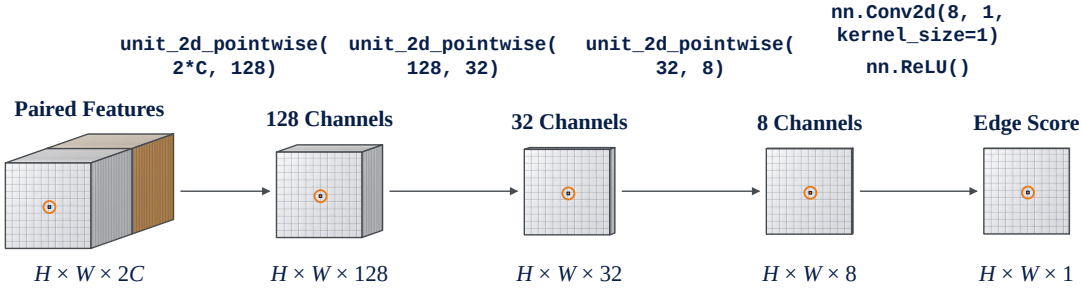


Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

DiscoGraph: Edge Scores



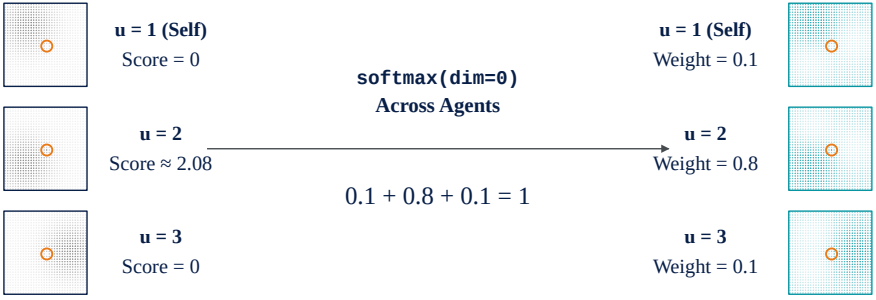
Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

DiscoGraph: Spatial Weights

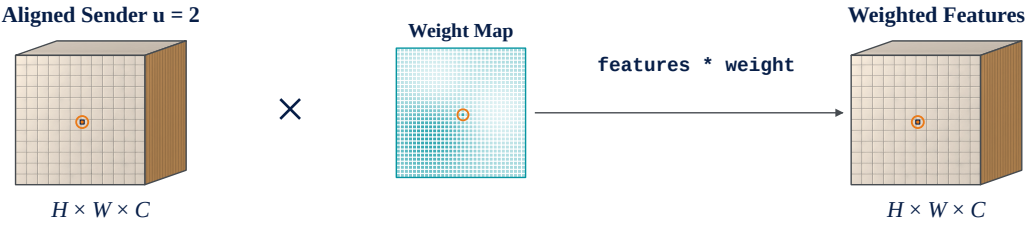
$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$



Source: AI generated illustration

DiscoGraph: Weighted Features

Multiply the aligned sender's entire C-channel feature vector by its weight at that cell.



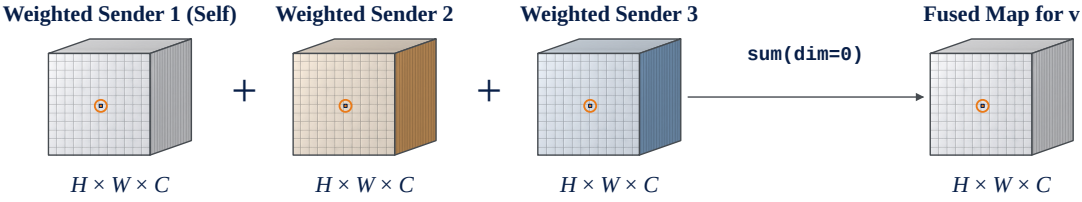
Tensor shape: $H \times W \times C$

Source: AI generated illustration

H = height, W = width, C = channels

DiscoGraph: Fused Features

Add the weighted contributions from all senders, including the receiver itself.

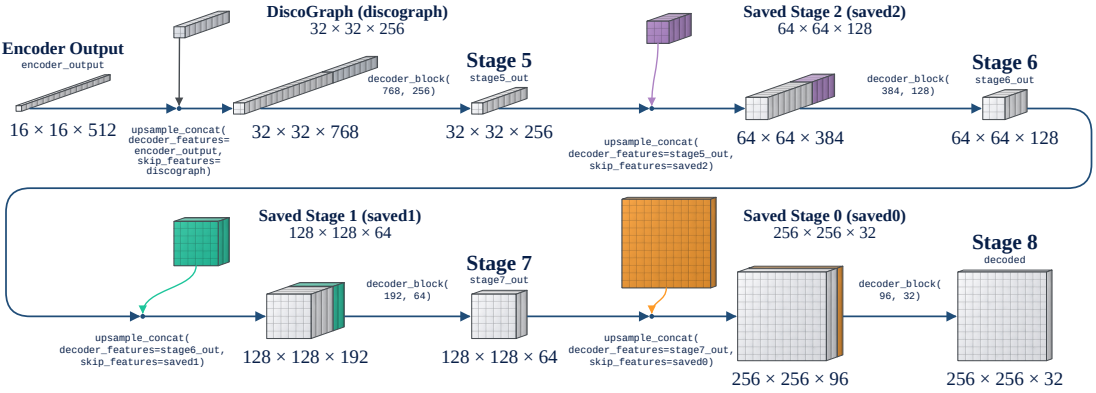


Tensor shape: $H \times W \times C$

Source: AI generated illustration

H = height, W = width, C = channels

Decoder Pipeline



Tensor shape: $H \times W \times C$

H = height, W = width, C = channels

Source: AI generated illustration

Appendix B

Evaluation and Reproduction

IoU, confidence scores, precision and recall, AP, dataset statistics, camera views, and benchmark results.

Intersection over Union (IoU)

Compare a predicted box with its ground-truth (reference) box, viewed from above.

$$\text{IoU} = \frac{\text{shared area (intersection)}}{\text{total area covered (union)}} = \frac{6}{8+8-6} = \frac{6}{10} = 0.60 = 60\%$$

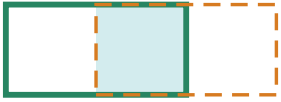
Each square is one area unit. IoU ranges from 0 (no overlap) to 1 (identical boxes).

Source: AI generated illustrations

IoU: Matching Thresholds

In the literature, IoU is usually associated with a matching threshold.

Example A: IoU = 0.33



Example B: IoU = 0.60

Example C: IoU = 0.80

Reference box (1) Predicted box (1)

Minimum IoU	A: 0.33	B: 0.60	C: 0.80
0.5 (50%)	Fail	Match	Match
0.7 (70%)	Fail	Fail	Match

Source: AI generated illustrations

DiscoNet Confidence Scores

1. Predict Raw Outputs **2. Convert With Softmax** **3. Keep and Rank Boxes**

$$\text{score} = \frac{e^{z_{\text{car}}}}{e^{z_{\text{background}}} + e^{z_{\text{car}}}}$$

Illustrative Example: Raw Outputs to Scores

Box	Background Logit	Car Logit	Car Score	Score > 0.7?
A	1.0	3.0	0.881 (88.1%)	Keep
B	2.0	1.0	0.269 (26.9%)	Discard
C	1.0	4.0	0.953 (95.3%)	Keep

Box A: Apply the Formula

$$\frac{e^3}{e^1 + e^3} \approx 0.881$$

88.1% car confidence

Source: AI generated illustration

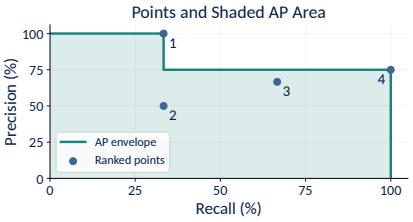
Precision, Recall, and AP

Rank by confidence (car score). TP = true positive (correct match), FP = false positive (unmatched/duplicate).

$$\text{Precision} = \frac{\text{TP}}{\text{TP+FP}} \qquad \text{Recall} = \frac{\text{TP}}{\text{reference cars}} \qquad \text{IoU} \geq 0.5$$

Illustrative Example: 3 Reference Cars

Top k	Score	Result	TP / FP So Far	Precision	Recall
1	0.99	TP	1 / 0	100.0%	33.3%
2	0.96	FP	1 / 1	50.0%	33.3%
3	0.92	TP	2 / 1	66.7%	66.7%
4	0.83	TP	3 / 1	75.0%	100.0%



$$\text{AP} = \frac{1}{3} \times 100\% + \frac{1}{3} \times 75\% + \frac{1}{3} \times 75\% = \mathbf{83.3\%}$$

Source: AI generated illustration

V2X-Sim 2.0: Dataset Analysis

100 scenes, 10,000 synchronized frames and 37,200 vehicle scans; roadside unit excluded.

Official Partitions

Partition	Scenes	Frames	Scans	Points
Train	80	8,000	29,200	1.0 B
Validation	10	1,000	4,000	106.8 M
Test	10	1,000	4,000	100.8 M
Total	**100**	**10,000**	**37,200**	**1.2 B**

One scan = one vehicle at one frame.

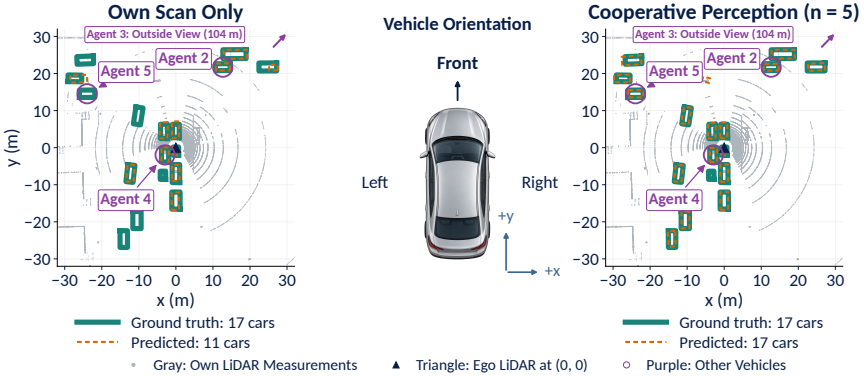
Grouped by Vehicle Inputs

Partition	Inputs	Scenes	Frames	Scans	Points
Train	2	4	400	800	28.9 M
	3	37	3,700	11,100	375.4 M
	4	22	2,200	8,800	294.8 M
	5	17	1,700	8,500	333.9 M
Validation	3	3	300	900	22.0 M
	4	4	400	1,600	48.4 M
	5	3	300	1,500	36.4 M
Test	3	3	300	900	22.7 M
	4	4	400	1,600	41.0 M
	5	3	300	1,500	37.1 M

Source: AI generated illustrations

V2X-Sim 2.0: Car Detections

Example 1: scene_5, raw frame 000066; vehicle 1, five participating vehicles.



Front is $+y$, right is $+x$; $(0, 0)$ is the LiDAR sensor. References can include cars hidden from its scan.

Source: AI generated illustration

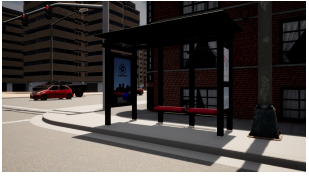
V2X-Sim 2.0: Camera Views

Six synchronized views provide context; the detector uses LiDAR. Camera and LiDAR positions differ.

Front Left

Front

Front Right



Rear Left

Rear

Rear Right

Source: V2X-Sim 2.0

Experiment Setup

Configuration of the completed 1,000-frame benchmark.

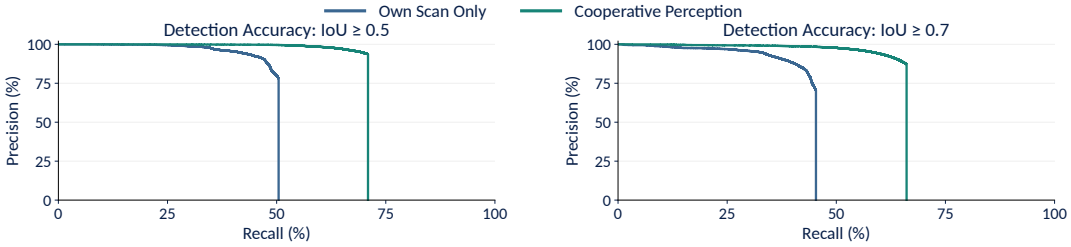
Hardware	
GPU	NVIDIA GeForce RTX 5060 Ti
GPU Memory	16 GB
CPU	AMD Ryzen 7 5700X
CPU Cores	8 cores / 16 threads
System RAM	16 GB

Software	
OS	Omarchy (Linux)
Python	3.14.8
PyTorch	2.9.1+cu128
CUDA Runtime	12.8

Source: AI generated illustrations

V2X-Sim 2.0: Reproduced Results

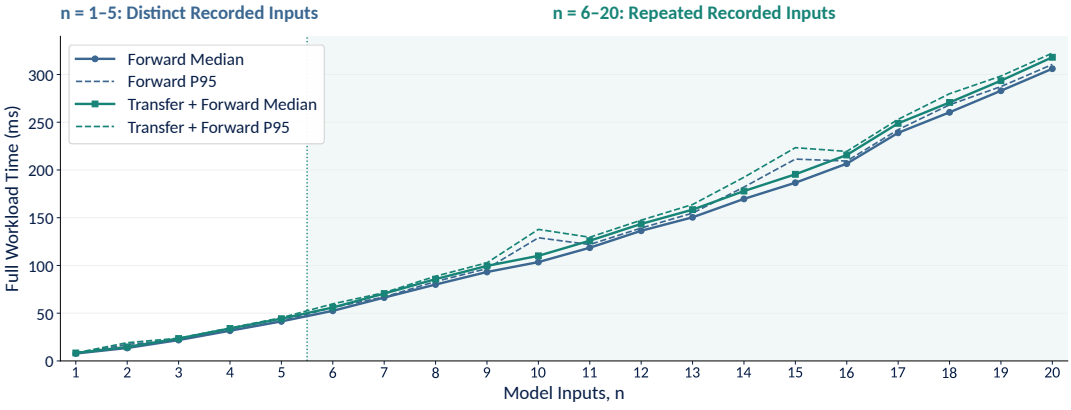
Condition	AP@0.5 (%)	AP@0.7 (%)
Own scan only	49.10	43.05
Cooperative perception	**70.41**	**64.82**



Own scan only switches information sharing off; both conditions use the same weights.
Sharing improves AP by 21.31 and 21.77 percentage points, respectively.

Source: AI generated illustrations

V2X-Sim 2.0: Runtime Scaling



Source: AI generated illustration

Appendix C

Conv2D and Conv3D

The complete numerical walkthrough: input tensor, kernels, and every spatial and temporal sliding window.

Initial Tensor

Assumptions

Batch size

$N = 1$

Channels $C_{\text{in}} \rightarrow C_{\text{out}}$

$3 \rightarrow 1$

Video $D \times H \times W$

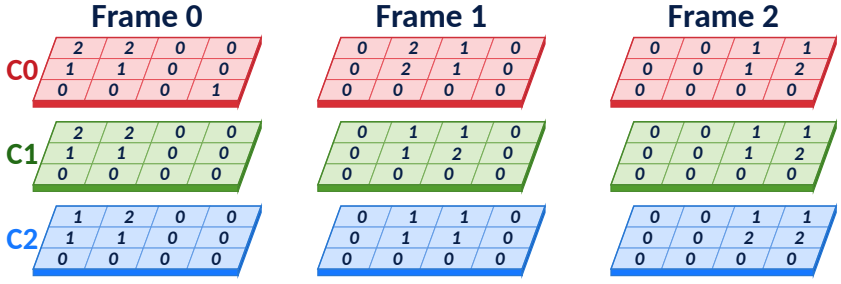
$3 \times 3 \times 4$

PyTorch defaults

Groups	1
Stride	1
Padding	0
Dilation	1

Bias = **0**
Set for this example

Consider the following video tensor

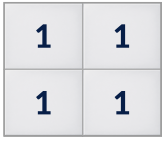


In this example, **Conv2D** uses the first frame; **Conv3D** uses the whole tensor

Kernels

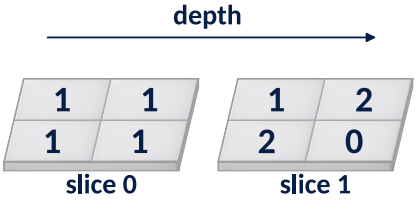
Use the shown weights in each of the three input channels. Bias $= 0$

Conv2D



Kernel size: $(2, 2)$

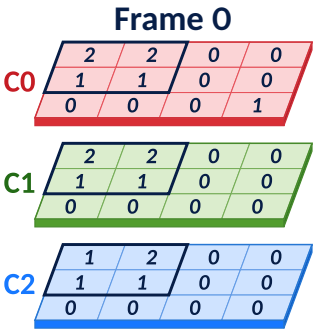
Conv3D



Kernel size: $(2, 2, 2)$

Conv2D: Sliding Window

Conv2D at $(r, c) = (0, 0)$: apply the 2×2 kernel to each channel of Frame 0



Kernel

1	1
1	1

Output

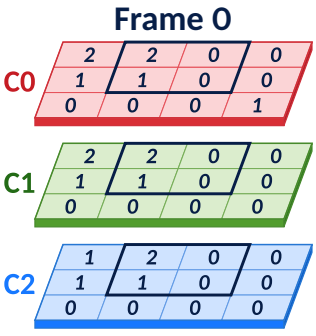
17	9	0
6	3	1

C0: $2 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 = 6$ **C1:** $2 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 = 6$

C2: $1 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 + 1 \cdot 1 = 5$ **Output:** $6 + 6 + 5 = \boxed{17}$

Conv2D: Sliding Window

Conv2D at $(r, c) = (0, 1)$: apply the 2×2 kernel to each channel of Frame 0



Kernel

1	1
1	1

Output

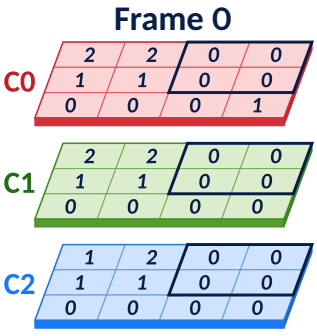
17	9	0
6	3	1

C0: $2 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 3$ **C1:** $2 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 3$

C2: $2 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 = 3$ **Output:** $3 + 3 + 3 = \boxed{9}$

Conv2D: Sliding Window

Conv2D at $(r, c) = (0, 2)$: apply the 2×2 kernel to each channel of Frame 0



Kernel

1	1
1	1

Output

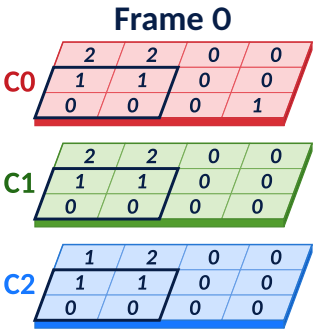
17	9	0
6	3	1

C0: $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 0$ **C1:** $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 0$

C2: $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 0$ **Output:** $0 + 0 + 0 = \boxed{0}$

Conv2D: Sliding Window

Conv2D at $(r, c) = (1, 0)$: apply the 2×2 kernel to each channel of Frame 0



Kernel

1	1
1	1

Output

17	9	0
6	3	1

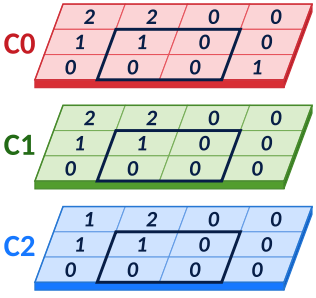
C0: $1 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 2$ **C1:** $1 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 2$

C2: $1 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 2$ **Output:** $2 + 2 + 2 = \boxed{6}$

Conv2D: Sliding Window

Conv2D at $(r, c) = (1, 1)$: apply the 2×2 kernel to each channel of Frame 0

Frame 0



Kernel

1	1
1	1

Output

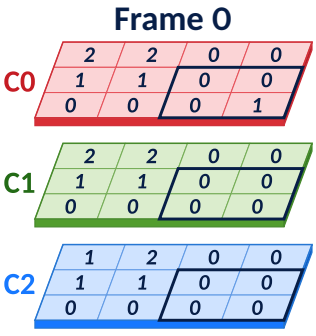
17	9	0
6	3	1

C0: $1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 1$ **C1:** $1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 1$

C2: $1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 1$ **Output:** $1 + 1 + 1 = \boxed{3}$

Conv2D: Sliding Window

Conv2D at $(r, c) = (1, 2)$: apply the 2×2 kernel to each channel of Frame 0



Kernel

1	1
1	1

Output

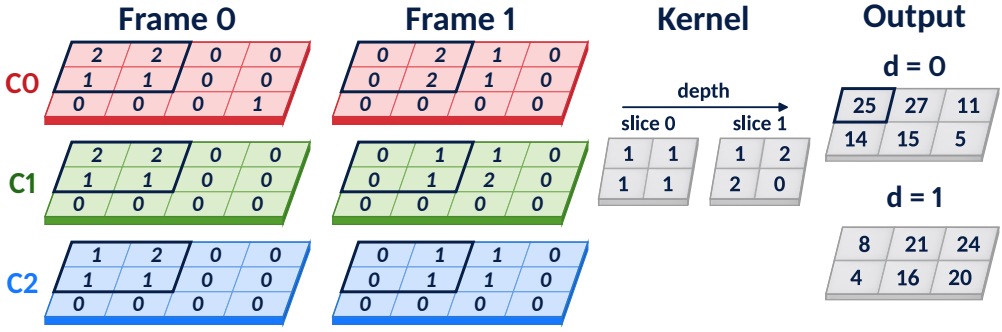
17	9	0
6	3	1

C0: $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 = 1$ **C1:** $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 0$

C2: $0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 = 0$ **Output:** $1 + 0 + 0 = \boxed{1}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 0, 0)$: use the same two-slice kernel in each channel

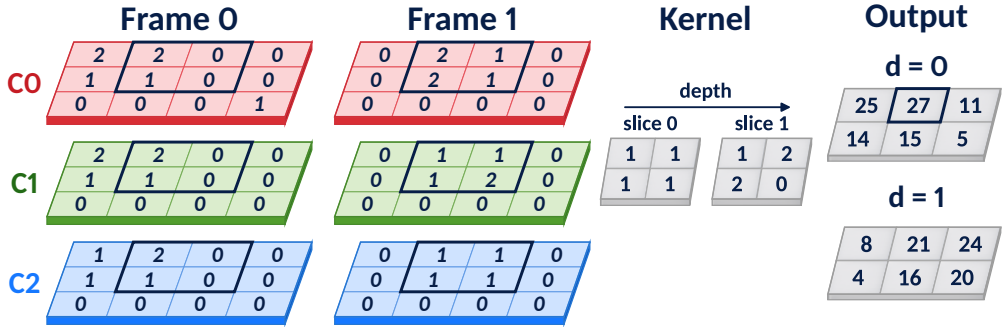


C0: $(2 \cdot 1 + 2 \cdot 1 + 1 \cdot 1 + 1 \cdot 1) + (0 \cdot 1 + 2 \cdot 2 + 0 \cdot 2 + 2 \cdot 0) = 6 + 4 = 10$

C1: $6 + 2 = 8$; **C2:** $5 + 2 = 7$ **Output:** $10 + 8 + 7 = \boxed{25}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 0, 1)$: use the same two-slice kernel in each channel

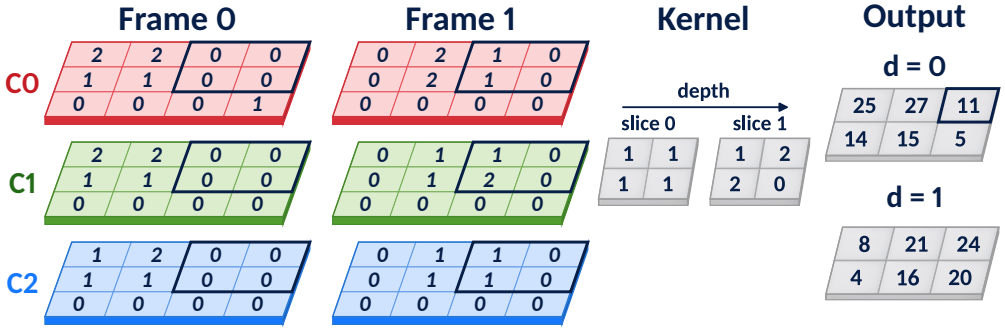


C0: $(2 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1) + (2 \cdot 1 + 1 \cdot 2 + 2 \cdot 2 + 1 \cdot 0) = 3 + 8 = 11$

C1: $3 + 5 = 8$; **C2:** $3 + 5 = 8$ **Output:** $11 + 8 + 8 = \boxed{27}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 0, 2)$: use the same two-slice kernel in each channel

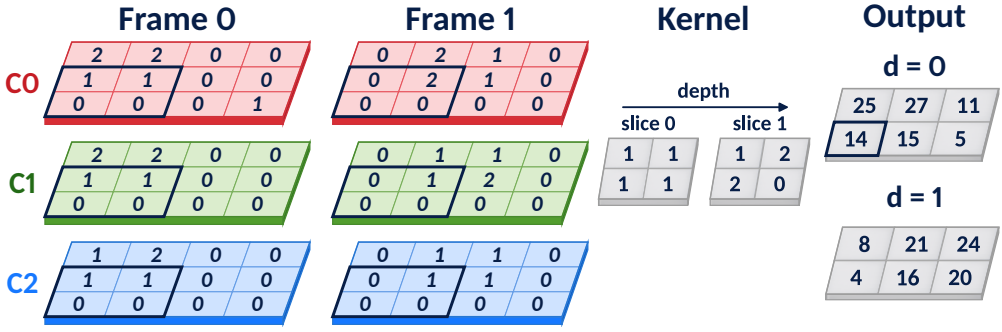


C0: $(0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (1 \cdot 1 + 0 \cdot 2 + 1 \cdot 2 + 0 \cdot 0) = 0 + 3 = 3$

C1: $0 + 5 = 5$; **C2:** $0 + 3 = 3$ **Output:** $3 + 5 + 3 = \boxed{11}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 1, 0)$: use the same two-slice kernel in each channel

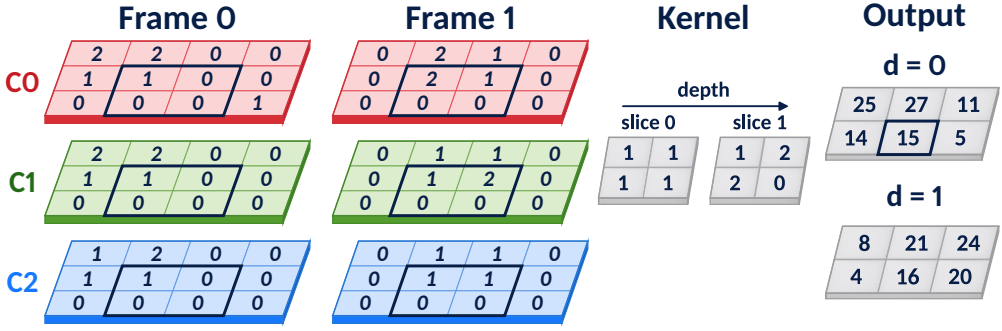


C0: $(1 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (0 \cdot 1 + 2 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 2 + 4 = 6$

C1: $2 + 2 = 4$; **C2:** $2 + 2 = 4$ **Output:** $6 + 4 + 4 = \boxed{14}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 1, 1)$: use the same two-slice kernel in each channel

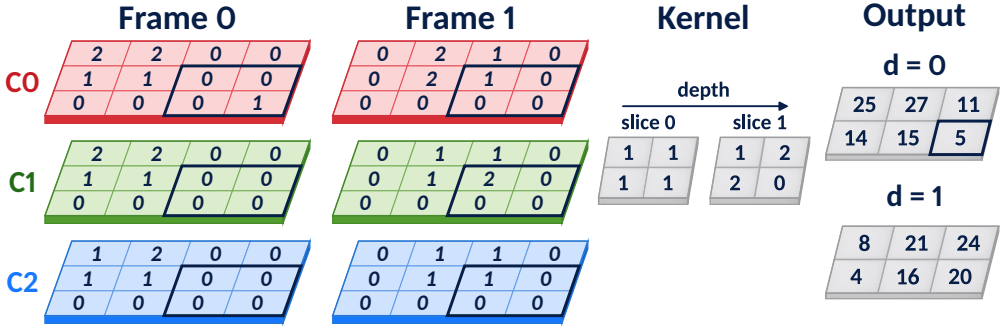


C0: $(1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (2 \cdot 1 + 1 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 1 + 4 = 5$

C1: $1 + 5 = 6$; **C2:** $1 + 3 = 4$ **Output:** $5 + 6 + 4 = \boxed{15}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (0, 1, 2)$: use the same two-slice kernel in each channel

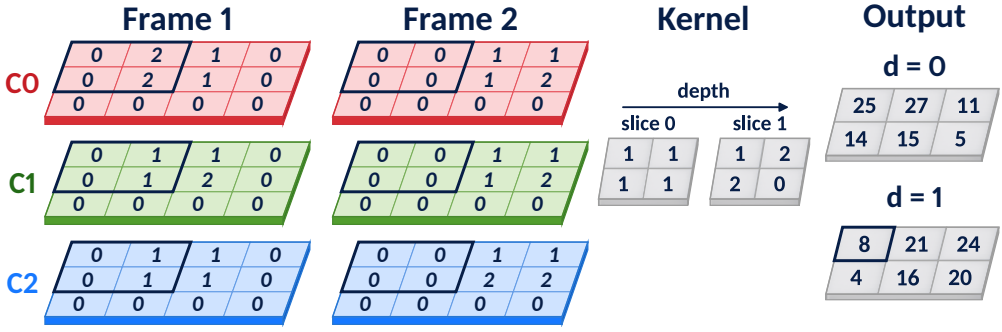


C0: $(0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 1 \cdot 1) + (1 \cdot 1 + 0 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 1 + 1 = 2$

C1: $0 + 2 = 2$; **C2:** $0 + 1 = 1$ **Output:** $2 + 2 + 1 = \boxed{5}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 0, 0)$: use the same two-slice kernel in each channel

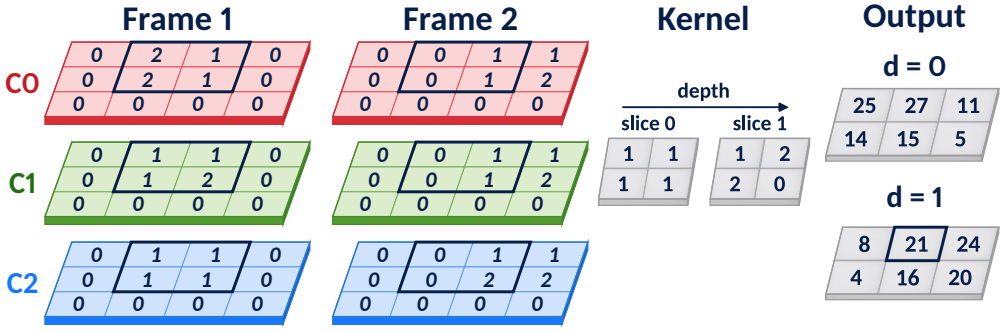


C0: $(0 \cdot 1 + 2 \cdot 1 + 0 \cdot 1 + 2 \cdot 1) + (0 \cdot 1 + 0 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 4 + 0 = 4$

C1: $2 + 0 = 2$; **C2:** $2 + 0 = 2$ **Output:** $4 + 2 + 2 = \boxed{8}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 0, 1)$: use the same two-slice kernel in each channel

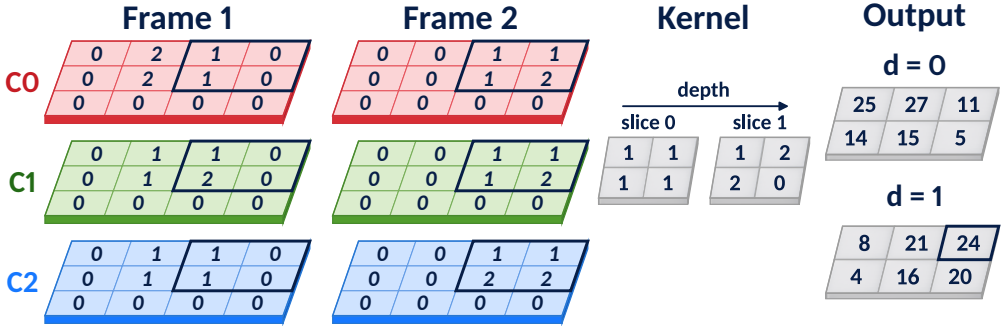


C0: $(2 \cdot 1 + 1 \cdot 1 + 2 \cdot 1 + 1 \cdot 1) + (0 \cdot 1 + 1 \cdot 2 + 0 \cdot 2 + 1 \cdot 0) = 6 + 2 = 8$

C1: $5 + 2 = 7$; **C2:** $4 + 2 = 6$ **Output:** $8 + 7 + 6 = \boxed{21}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 0, 2)$: use the same two-slice kernel in each channel

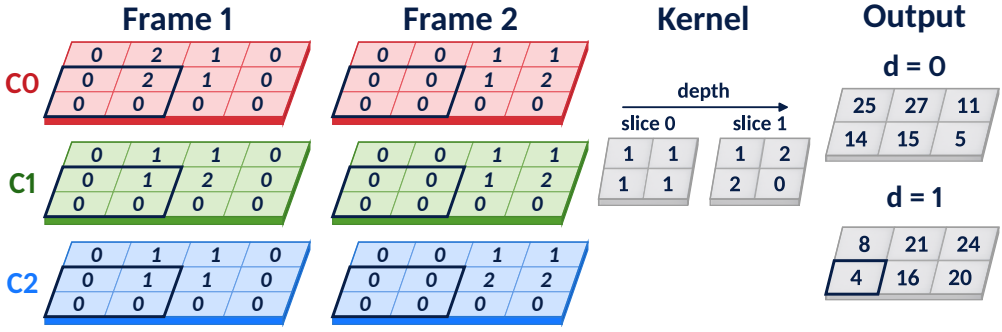


C0: $(1 \cdot 1 + 0 \cdot 1 + 1 \cdot 1 + 0 \cdot 1) + (1 \cdot 1 + 1 \cdot 2 + 1 \cdot 2 + 2 \cdot 0) = 2 + 5 = 7$

C1: $3 + 5 = 8$; **C2:** $2 + 7 = 9$ **Output:** $7 + 8 + 9 = \boxed{24}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 1, 0)$: use the same two-slice kernel in each channel

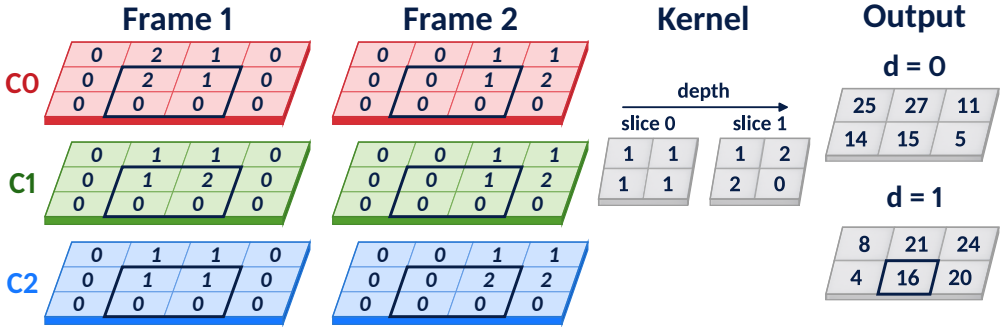


C0: $(0 \cdot 1 + 2 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (0 \cdot 1 + 0 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 2 + 0 = 2$

C1: $1 + 0 = 1$; **C2:** $1 + 0 = 1$ **Output:** $2 + 1 + 1 = \boxed{4}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 1, 1)$: use the same two-slice kernel in each channel

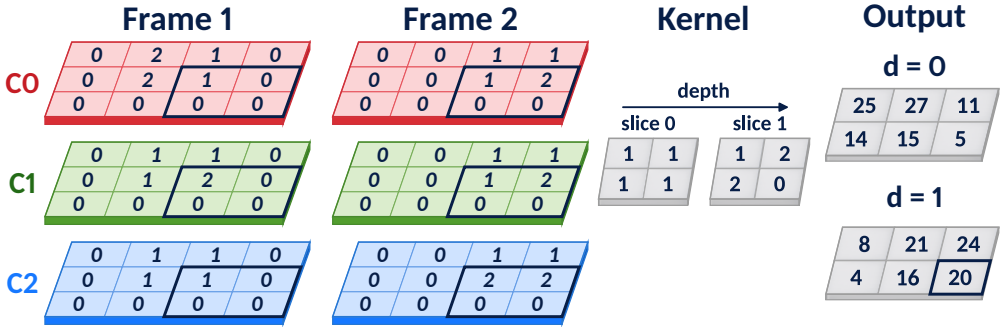


C0: $(2 \cdot 1 + 1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (0 \cdot 1 + 1 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 3 + 2 = 5$

C1: $3 + 2 = 5$; **C2:** $2 + 4 = 6$ **Output:** $5 + 5 + 6 = \boxed{16}$

Conv3D: Sliding Window

Conv3D at $(d, r, c) = (1, 1, 2)$: use the same two-slice kernel in each channel



C0: $(1 \cdot 1 + 0 \cdot 1 + 0 \cdot 1 + 0 \cdot 1) + (1 \cdot 1 + 2 \cdot 2 + 0 \cdot 2 + 0 \cdot 0) = 1 + 5 = 6$

C1: $2 + 5 = 7$; **C2:** $1 + 6 = 7$ **Output:** $6 + 7 + 7 = \boxed{20}$

Appendix D

Complete Experiment Record

All 20 report pages: recorded examples, training and test protocol, losses, runtime scaling, and Original/Tree results.

DiscoNet: Our First Experiment

Raw Scene ID: scene_5 · Raw Frame ID: 000066 · Agents: 1 (ego), 2, 3, 4, 5
Full Frame: 152 annotated vehicles · 0 other entities · 152 total (including all 5 agents)
110 cars · 8 police cars · 12 motorcycles · 22 bicycles

PURPOSE

Check that the trained model runs, and see how its car predictions change when vehicles share information.

WHAT WE COMPARED

Own scan only: each vehicle uses its own LiDAR measurements. Sharing with other vehicles is off.

With sharing: the same model also uses information extracted from the other vehicles' LiDAR scans.

RESULT FOR EGO VEHICLE 1

11
Own scan only
cars predicted

17
With sharing
cars predicted

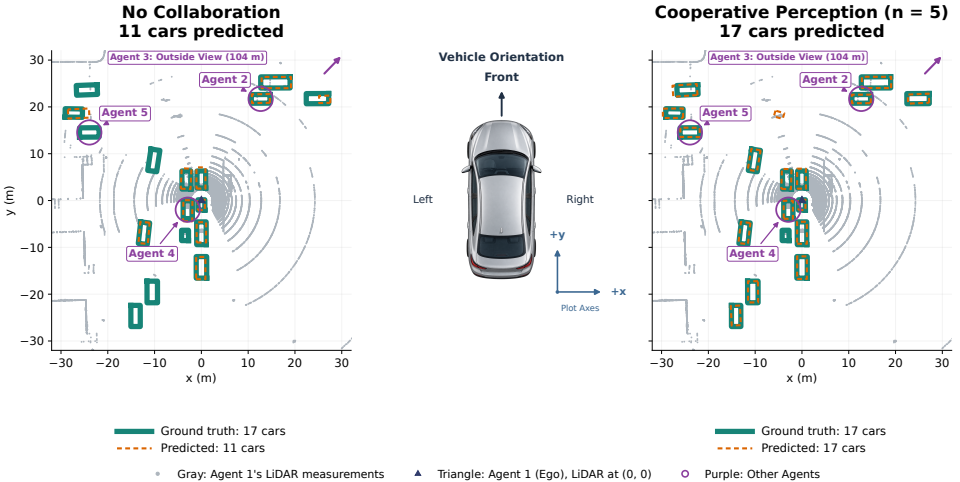
TAKEAWAY

Reference car boxes matched:
10 with own scan only
16 with cooperation

At 50% box overlap (IoU ≥ 0.5).

Example 1: Reference and Predicted Car Boxes

Plot axes: +x right, +y forward; (0, 0) is the LiDAR sensor.
Wide solid green: dataset reference (ground truth). Thin dashed orange: model predictions.



The dataset labels can include cars hidden from this vehicle's sensor.

AI generated illustration

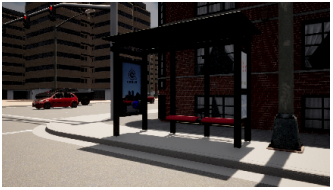
Example 1: Camera Views

These six camera views were recorded at the same moment as the LiDAR scan in our experiment.

Front Left

Front

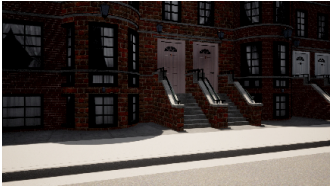
Front Right



Rear Left

Rear

Rear Right



The images provide visual context; this detector used LiDAR measurements. Camera and LiDAR positions differ.

Original simulated RGB images · 1600 × 900 pixels each · V2X-Sim 2.0 · scene_5, frame 60

V2X-Sim 2.0: Dataset Overview

Recorded virtual traffic for studying how vehicles perceive the same scene together

Simulated Traffic

Created with CARLA and SUMO.
The saved recordings can be used without running the simulators.

Synchronized Vehicle Views

Vehicles and a roadside unit observe the same moment together.
Each agent has a different viewpoint.

Sensors and Reference Labels

LiDAR, camera images and vehicle poses.
Labels identify objects in the scene.
Supports detection, tracking and segmentation.

From Recorded Point Clouds to Our Model Input

Original Dataset

LiDAR point clouds and camera images, with vehicle poses and reference labels.

Preprocess →

Prepared Detection Release

LiDAR converted into bird's-eye-view grids.
The grids record which 3D cells are occupied.
We used the authors' prepared detection files.

Official scene split: 80 Train · 10 Validation · 10 Test

Our benchmark starts from the prepared vehicle LiDAR grids.

DiscoNet's encoder still computes the learned features from these grids.

Camera images provide visual context; the roadside unit is excluded from our model inputs.

Sources: V2X-Sim dataset website CoPerception data preparation

AI generated illustration

Authors' Training Setup and Dataset Splits

Published V2X-Sim 2.0 protocol for the released DiscoNet student · Roadside unit excluded

Partition	Scenes	Synchronized Frames	Vehicle Inputs per Scene	Vehicle Scans	Raw LiDAR Points
Train	80	8,000	2–5	29,200	1.0B
Validation	10	1,000	3–5	4,000	106.8M
Test	10	1,000	3–5	4,000	100.8M
Total	**100**	**10,000**	**—**	**37,200**	**1.2B**

Train: learn weights. Validation: held-out development data. Test: final accuracy evaluation.

Released Checkpoint: Epoch 100

The authors train the DiscoNet student using a teacher model; the student is used for inference.

Our experiment loads those trained weights and evaluates the 10 test scenes; no retraining.

The next table aggregates each partition by the number of vehicle inputs.

One dataset pass; raw returns before filtering/cropping. M = million; B = billion (one decimal); exact counts are in the CSV.

Published code and splits describe the protocol; the checkpoint does not contain a scene-by-scene training history.

Sources: Authors' split lists Training code / recipe Released checkpoint

Dataset Statistics by Partition and Vehicle Inputs

V2X-Sim 2.0 · One row per (partition, vehicle inputs) · Counts summed across scenes

Partition	Vehicle Inputs	Scenes	Total Frames	Total Vehicle Scans	Total Raw LiDAR Points
Train	2	4	400	800	28.9M
Train	3	37	3,700	11,100	375.4M
Train	4	22	2,200	8,800	294.8M
Train	5	17	1,700	8,500	333.9M
Validation	3	3	300	900	22.0M
Validation	4	4	400	1,600	48.4M
Validation	5	3	300	1,500	36.4M
Test	3	3	300	900	22.7M
Test	4	4	400	1,600	41.0M
Test	5	3	300	1,500	37.1M
Total	—	**100**	**10,000**	**37,200**	**1.2B**

Authors' Epoch-100 DiscoNet Student · Our Run: Test Evaluation with Fixed Weights

Vehicle inputs = nonempty prepared vehicle records per frame; roadside-unit inputs are excluded.

Frames, vehicle scans and raw LiDAR points are totals over every scene in each row; one dataset pass.

M = million; B = billion (rounded to one decimal). Raw points precede filtering/cropping; exact counts remain in the CSV.

Partition assignments follow the authors' published splits; the checkpoint does not store its scene-by-scene training history.

Sources: Authors' split lists Training code / recipe Released checkpoint

Experiment Setup: Hardware and Software

Configuration of the completed 1,000-frame benchmark

Hardware	
GPU	NVIDIA GeForce RTX 5060 Ti
GPU Memory	16 GB
CPU	AMD Ryzen 7 5700X
CPU Cores	8 cores / 16 threads
System RAM	16 GB

Software	
OS	Omarchy (Linux)
Python	3.14.8
PyTorch	2.9.1+cu128
CUDA Runtime	12.8

Model and Execution

Official DiscoNet student · Epoch 100 · Feature sharing at layer 3 · Roadside unit excluded

Single GPU · float32 precision · One synchronized frame per batch · Seed 0 · Inference only

Dataset: V2X-Sim 2.0, official prepared detection test split · 10 scenes · 1,000 frames

Experiment Procedure, Metrics, and Running Time

Offline evaluation on the given dataset; CARLA was not running

1. Read One Synchronized Frame

Load the stored LiDAR voxel grids
from 3–5 active vehicles.
Process one frame at a time.

2. Run the Same Model Twice

No Collaboration: own scan only.
Cooperative Perception: sharing on.
Use the same trained weights.

3. Measure Each Vehicle's Output

Each active agent predicts car boxes.
Match to that agent's reference boxes
in that agent's own coordinates.

1,000 Frames → 4,000 Vehicle Views → Pooled AP Scores

One vehicle view = one active agent at one frame. Match within each view, then pool all views for scoring.

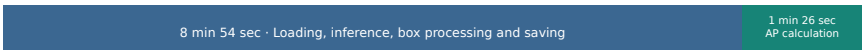
The 50,047 reference car boxes count each agent's view separately; the same car can appear in multiple views.

AP (Average Precision) summarizes precision (% of predictions correct) and recall (% of reference cars found).

IoU (Intersection over Union) = overlapping box area ÷ combined area. Report AP@0.5 and AP@0.7.

Keep confidence > 0.7; remove overlapping duplicates using Non-Maximum Suppression (NMS), IoU 0.01.

Running Time for Both Conditions Together



10 min 20 sec

Total

Whole test split; excludes dataset download, initial model loading and figure export.

AI generated illustration

Original Test Split: Loss and Detection Accuracy

Authors' epoch-100 checkpoint · 10 test scenes · 1,000 frames · 4,000 vehicle views

Condition	Classification Loss ↓	Box-Localization Loss ↓	Total Loss ↓	AP@0.5 (%) ↑	AP@0.7 (%) ↑
No Collaboration	3.0K	13.8K	16.8K	49.10	43.05
Cooperative Perception	1.5K	9.6K	11.1K	70.41	64.82

K = thousand; loss values are rounded to one decimal. Exact values remain in the source CSV.

Loss Measures Model Error; Lower Is Better

Classification loss scores car/background decisions. Box-localization loss scores errors in predicted box corners.

Total loss = classification + box-localization loss, using the authors' original test functions and normalization.

Detection Quality Is Measured by AP; Higher Is Better

AP = Average Precision. IoU = box overlap; AP@0.7 requires tighter box alignment than AP@0.5.

Both conditions use the same weights. No Collaboration uses each vehicle's own scan with information sharing off.

Authors' checkpoint log: 70.4136% / 64.8239% AP. Our cooperative AP matches to two decimals.

Additional loss-evaluation pass: 2 min 31 sec. AP and its curves come from our original full-test run.

Fixed checkpoint; no weight updates. Test loss is averaged over synchronized frames using the authors' test-code normalization.

How DiscoNet Computes Confidence Scores

Confidence and score refer to the same car-class output · Both inference modes use the same calculation

1. Predict Raw Outputs

Each logit = weighted head features + bias.
Two outputs per preset box (anchor):
one for background and one for car.

2. Convert With Softmax

Convert the logits into estimated background/car probabilities summing to 1.
Use the car probability as the score.

3. Keep and Rank Boxes

Keep scores > 0.7 and remove duplicates.
Prefer higher scores when boxes overlap.
Rank retained scores to build the AP curve.

$$\text{score} = \frac{e^{z_{car}}}{e^{z_{background}} + e^{z_{car}}}$$

z is a raw output (logit), before softmax.
The car score lies between 0 and 1.

Illustrative Example: Raw Outputs to Scores

Box	Background Logit	Car Logit	Car Score	Score > 0.7?
A	1.0	3.0	0.881 (88.1%)	Keep
B	2.0	1.0	0.269 (26.9%)	Discard
C	1.0	4.0	0.953 (95.3%)	Keep

Box A: Apply the Formula

$$\frac{e^3}{e^1 + e^3} \approx 0.881$$

88.1% car confidence

If A and C survive duplicate removal:
C (0.953) ranks before A (0.881).

Reference boxes determine correctness during evaluation. A high-confidence detection can still be a false alarm.

Training uses car/background labels to learn the head's weights; our evaluation keeps those weights fixed.

Authors' Classification Head

Authors' Softmax Scoring

AI generated illustration

How Precision–Recall Curves and AP Are Computed

AP = Average Precision · 4,000 views per condition · One view = one vehicle in one frame

1. Rank Predicted Boxes

Pool all retained predicted boxes.
Sort by confidence, highest first.
Include one more box at each step.

2. Match Within Each View

TP (true positive) = correct car match.
FP (false positive) = unmatched/duplicate.
One match per car; IoU ≥ 0.5 or ≥ 0.7.

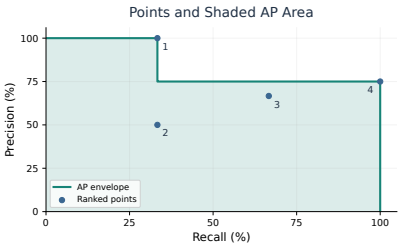
3. Build the Curve and Its Area

Recompute precision and recall.
Best precision at this or higher recall.
AP = area under this precision envelope.

Precision = TP / (TP + FP) **Recall = TP / Number of Reference Cars**

Illustrative Example: 3 Reference Cars

Top k	Score	Result	TP / FP So Far	Precision	Recall
1	0.99	TP	1 / 0	100.0%	33.3%
2	0.96	FP	1 / 1	50.0%	33.3%
3	0.92	TP	2 / 1	66.7%	66.7%
4	0.83	TP	3 / 1	75.0%	100.0%



Prediction 2 is a false alarm: precision falls; recall stays at 33.3%.

Example AP = (⅓ × 100%) + (⅓ × 75%) + (⅓ × 75%) = 83.3%

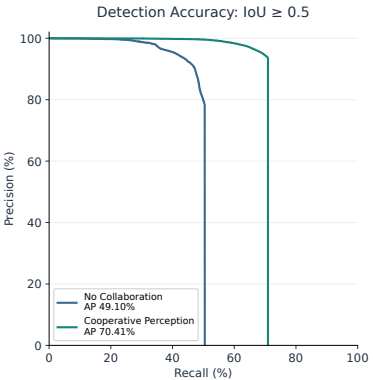
Repeat for No Collaboration and Cooperative Perception at each IoU threshold.
Real curves use the authors' score > 0.7 and duplicate-removal filters; missed cars limit recall.

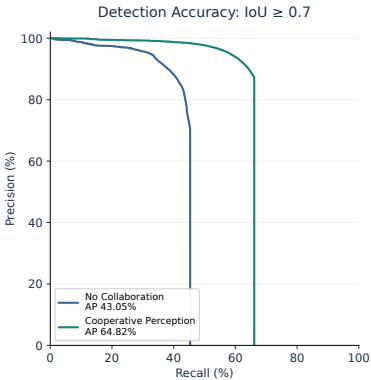
Calculation: Authors' AP code

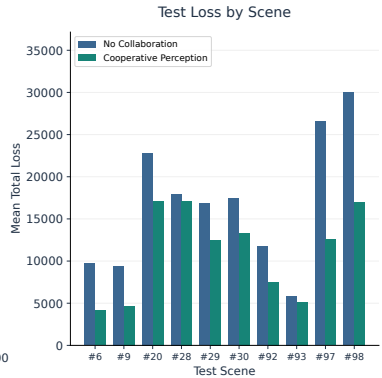
AI generated illustration

Original Test Split: Accuracy and Loss Plots

One fixed checkpoint evaluated on the authors' original test split · Information sharing off versus on







Precision: fraction of predictions correct. Recall: fraction of reference cars found. Higher accuracy curves are better.

Loss bars average 100 frames per scene. Each scene is a separate recording; lower bars indicate lower model error.

AP pools 4,000 active vehicle views. Curves use the authors' confidence cutoff > 0.7 and duplicate-removal settings.

The loss bars describe test-set evaluation of fixed weights. The checkpoint contains no epoch-by-epoch training history.

Custom Experiments

Runtime Versus Model Input Count

V2X-Sim 2.0 · n = 1–20

n = 1–5: Distinct Recorded Inputs

n = 6–20: Replayed-Input Workloads

Same trained checkpoint · One GPU · Runtime only · n counts model input slots

Custom Experiment 1: V2X-Sim Inputs and Replay

Measure runtime through n = 20 while keeping the source dataset and trained checkpoint fixed

Same Recorded Frames at Every n

Raw Scene	Frames	Recorded Vehicles
scene_9	10	5
scene_97	10	5
scene_98	10	5
Total	30	5 per frame

Vehicle IDs: 1 (ego), 2, 3, 4, 5 · No roadside inputs.

30 synchronized frames · 150 verified source files.

n Counts Model Input Slots

n = 1–5: use distinct recorded vehicle inputs.
Keep ego 1 and add the nearest recorded vehicles.
n = 6–20: cycle the five recorded inputs again.
Reuse each input's recorded pairwise poses.
Allocate n inputs and n active model slots.

Example: n = 20

Vehicle inputs [1, 2, 3, 4, 5] repeated four times.
Five recorded vehicles supply twenty model slots.

Inputs are the authors' prepared occupancy grids, derived from synchronized V2X-Sim 2.0 LiDAR recordings.

Above five, this is a runtime workload test with repeated inputs; it does not add newly recorded vehicles.

All n values are timed again on this same 30-frame cohort using unchanged epoch-100 weights.

Source: Original prepared V2X-Sim 2.0 detection test split and the verified runtime-workload record

Custom Experiment 1: Measuring Runtime

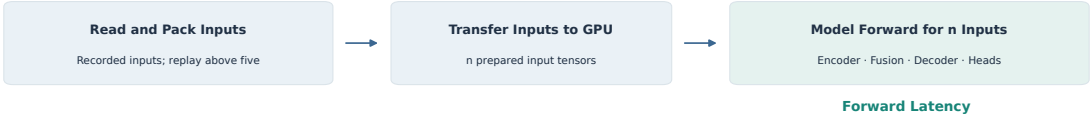
NVIDIA GeForce RTX 5060 Ti · Same checkpoint · Float32 · One full n-input group per measurement

Time the Entire Model Workload

Forward covers the encoder, pairwise feature fusion, decoder and detection heads for all n input slots.
Start with input tensors already on the GPU.
Synchronize CUDA before and after the forward.
Report median inference time for the complete group.

Repeat Every Condition

Same 30 frames at every n from 1 through 20.
3 trials × 30 frames = 90 timing samples per n.
20 warmup forwards before each n in every trial.
Shuffle condition order between trials with seed 0.
1,800 timed forwards + 1,200 warmup forwards.



Median is the middle timing value across the 90 measured frames and trials for each n.

File reads, CPU packing, raw LiDAR preprocessing, warmups, model loading and output checks are outside the forward timer.

Runtime only: AP and box postprocessing are not computed. Wireless transmission is outside the measurement.

Custom Experiment 1: Runtime Results for n = 1–10

V2X-Sim 2.0 · Same 30 frames · Three trials · 90 timing samples per n · n counts model input slots

n Model Inputs	Recorded Vehicles	Replayed Inputs	Forward Median (ms)
1	1	0	7.8
2	2	0	13.6
3	3	0	21.8
4	4	0	31.6
5	5	0	41.4
6	5	1	52.5
7	5	2	66.3
8	5	3	80.0
9	5	4	93.2
10	5	5	103.5

Recorded Vehicles counts distinct source scans; Replayed Inputs counts additional copies of those scans.

All rows use n active model slots on one GPU. n = 6–20 is a replayed-input runtime workload.

Complete n = 1–20 sweep: 7 min 29 sec after device setup · Runtime only; no AP.

Median forward time includes encoder, collaboration, decoder and heads; input transfer, file reads and CPU packing are excluded.

Custom Experiment 1: Runtime Results for n = 11–20

V2X-Sim 2.0 · Same 30 frames · Three trials · 90 timing samples per n · n counts model input slots

n Model Inputs	Recorded Vehicles	Replayed Inputs	Forward Median (ms)
11	5	6	118.6
12	5	7	136.3
13	5	8	150.4
14	5	9	169.7
15	5	10	186.6
16	5	11	206.6
17	5	12	238.9
18	5	13	260.4
19	5	14	283.0
20	5	15	306.0

Recorded Vehicles counts distinct source scans; Replayed Inputs counts additional copies of those scans.

All rows use n active model slots on one GPU. n = 6–20 is a replayed-input runtime workload.

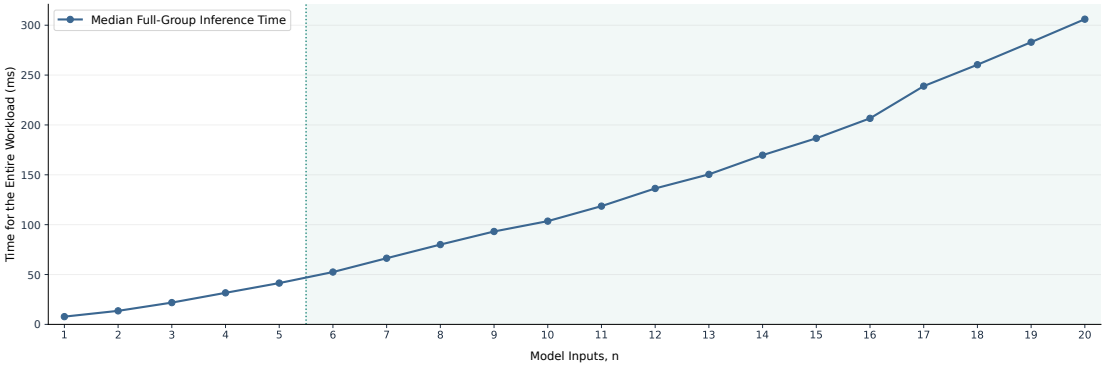
Complete n = 1–20 sweep: 7 min 29 sec after device setup · Runtime only; no AP.

Median forward time includes encoder, collaboration, decoder and heads; input transfer, file reads and CPU packing are excluded.

Custom Experiment 1: Runtime Versus Model Input Count

V2X-Sim 2.0 · 30 shared frames · NVIDIA GeForce RTX 5060 Ti · All n = 1–20 conditions measured

n = 1–5: Distinct Recorded Inputs **n = 6–20: Replayed-Input Workloads**



n = 1 → 20: forward median 7.8 → 306.0 ms (39.4×).

Each point is the median complete-forward time across 90 samples, including encoder, fusion, decoder and heads.

Above five inputs, the workload repeats the same five recorded vehicle scans and their poses. No AP is evaluated.

Source: Verified V2X-Sim runtime-workload study · All 1,800 timing samples and source mappings retained in the record

Custom Experiment 2: Coarse Attention and Kruskal Fusion

Compare the original complete graph with a selected tree · Same trained checkpoint · No retraining

Select Cheaply, Then Collaborate at Full Resolution

1. Coarse Attention

Pool each 32 × 32 feature map to 4 × 4.
Align and score every receiver–sender pair.
Normalize over senders at each cell.
Reduce each map to mean squared attention.

2. Maximum Spanning Tree

Average the two directional edge scores.
Kruskal keeps the highest-scoring links
without cycles, until n − 1 remain.
Use both directions of each selected link.

3. Sparse Collaboration

Keep each vehicle's own feature map.
Use original 32 × 32 alignment and scoring
only for its selected tree neighbors.
Renormalize weights, then decode as before.

Full-Resolution Pair Evaluations

Original: n^2 pairs. Tree: n self pairs + 2(n − 1) directed links = 3n − 2.
At n = 15: 225 → 43 fine pairs, plus 225 coarse pairs in one small batch.

Matched Runtime and Accuracy Evaluation

Same 30 frames · Fixed ego 1 · 90 timings per method and n.
Time the full model, including coarse scoring and Kruskal.
AP uses original ego labels, confidence > 0.7 and NMS IoU 0.01.

Only the full-resolution collaboration work becomes linear in n. Coarse pair scoring still scales with n^2.

One fusion round: each vehicle receives its immediate tree neighbors; information does not cross the whole tree in this pass.

n = 1–5 uses distinct recorded V2X-Sim inputs. n = 6–15 repeats the five recorded inputs and poses; AP describes that replay workload.

Source: Verified 30-frame original/tree experiment · Unchanged epoch-100 checkpoint

Custom Experiment 2: Runtime and Detection AP

NVIDIA GeForce RTX 5060 Ti · Same 30 ego views · 3 timing trials · Unchanged checkpoint · n = 1–15

Runtime: Median Full-Group Inference Time · AP: Average Precision at Ego 1

n	Original Median (ms)	Tree Median (ms)	Original AP@0.5 (%)	Tree AP@0.5 (%)	Original AP@0.7 (%)	Tree AP@0.7 (%)
1	7.82	7.86	48.91	48.91	45.54	45.54
2	13.34	13.25	60.94	60.94	53.67	53.67
3	21.99	21.41	66.52	60.17	58.71	52.34
4	31.39	29.38	70.80	58.06	63.39	50.37
5	41.57	36.54	74.46	62.65	67.53	53.66
6	52.43	43.63	73.54	57.71	66.61	50.69
7	66.67	52.89	73.93	57.87	66.70	50.57
8	79.83	60.47	73.80	59.16	66.90	52.01
9	92.07	67.48	74.50	61.56	67.06	53.33
10	102.80	70.12	74.46	61.72	67.53	53.66
11	118.76	76.93	74.00	58.57	67.05	50.44
12	133.49	82.84	74.17	58.77	67.20	50.72
13	147.97	86.52	74.36	59.50	67.18	51.61
14	165.79	92.73	74.90	60.93	67.50	52.75
15	185.05	99.25	74.46	60.48	67.53	52.45

At n = 15: 46.4% less inference time; AP changes -13.98 points at IoU 0.5 and -15.08 at IoU 0.7.

Blue rows: distinct recorded inputs (n ≤ 5). Green rows: five recorded inputs replayed into n model slots (n > 5).

Lower runtime and higher AP are better. n = 1 and 2 have identical graphs and predictions; their timing differences are measurement variation.

90 timing samples per entry · 427 reference car boxes at ego 1 · Selection included; input transfer and postprocessing excluded.

Source: Saved matched original/tree timings and detections; every AP and median recomputed before publication

Appendix E

Image Generation Example

Codex CLI, the generation prompt, and the result.

Simple Workflow

I used OpenAI Codex CLI with my ChatGPT account.

Get Access: Use a ChatGPT Plus or Pro subscription, install Codex CLI, and choose **Sign in with ChatGPT**.

1. Describe the image in a prompt.

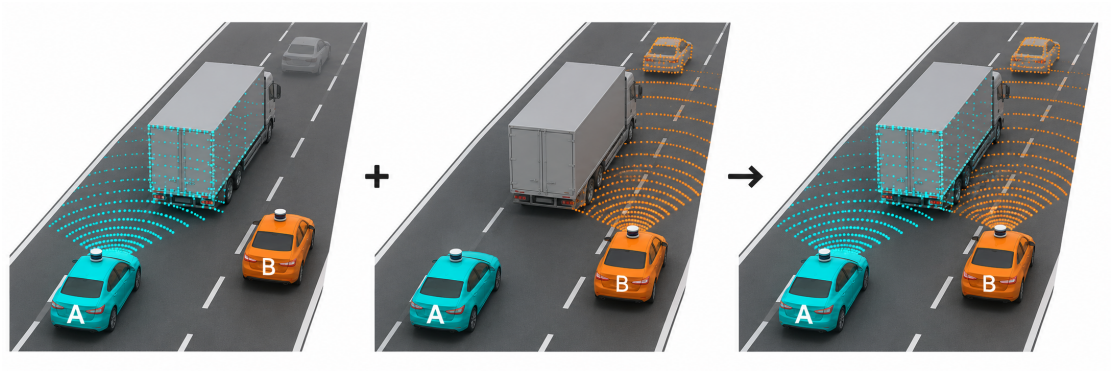
2. Put the generated image in the slide.

OpenAI documentation: Codex CLI and ChatGPT subscriptions.

Prompt: Generate the Image

Can u make a clear image with image-gen for the cooperative perception slide? I'd like three views side by side of the same scene, like a clean 3D illustration seen diagonally from above, on a white background. Car A is behind a big gray truck, and a smaller gray car is in front of that truck, hidden from A. Car B is in the next lane and can see around the truck. Give A a cyan color and B an orange one, with a little LiDAR on each roof. In the first view show only A's cyan points, in the second B's orange points reaching the small car, and in the last put the two together. Keep all four vehicles in the same positions in every view; the small car is just faint gray context in the first, with no cyan points on it. Put a plus and an arrow between the views. No slide title, trees, buildings or extra text, just A and B.

Generated Result



Result from the prompt.

Source: AI generated illustration.